

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП'ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

ЗВІТ
до лабораторної роботи №4
з дисципліни
«Технології проектування комп'ютерних систем»

Студента 4-го курсу
спеціальності 123 –
«Комп'ютерна інженерія»
Друмашка Костянтина

УЖГОРОД – 2026

Лабораторна робота № 4

Тема: Моделювання режиму очікування в цифрових пристроях за допомогою оператора wait

Мета: Отримати практичні навички застосування оператора wait при моделюванні різних режимів очікування у Active-HDL. Навчитися виконувати порівняння часових діаграм.

Варіант 11

Хід роботи

Присвоєння нових значень сигналам в процесах відбувається тільки після припинення процесу. Для задання умов припинення процесу використовується оператор wait. Якщо процес містить оператор wait, він виконує послідовно всі оператори до оператора wait. Після цього процес припиняється і очікує виконання умови, заданої в операторі wait. Як тільки це відбувається, процес поновлюється і виконує всі оператори, поки знову не зустріне оператор wait.

Існує три типи оператора wait:

- wait for *time_expression* - очікування на протязі певного часу;
- wait until *condition* - очікування виконання логічного виразу;
- wait on *sensitivity_list* - очікування зміни значення сигнала із списку:

Тип оператора wait	Опис
wait for <i>time_expression</i>	Припиняє процес на визначений час: час може бути заданий безпосередньо, або як вираз, результатом якого є значення часу.
wait until <i>condition</i>	Припиняє процес доти, доки задана умова не стане істинною завдяки зміні будь-якого сигналу, що містяться в умові. При цьому, якщо сигнал не змінюється, wait until не поновить процес, навіть якщо сигнал задовільняє умову.
wait on <i>sensitivity_list</i>	Припиняє процес доти, доки не відбудеться подія з будь-яким з сигналів, вказаних в списку чутливості.
складний wait	Містить комбінацію двох або трьох різних форм оператора wait.

Якщо симулятор знаходить оператор wait відразу на початку процесу, то процес буде негайно припинений і жодний оператор не буде виконаний. Якщо ж оператор wait розташований в кінці процесу, спочатку будуть виконані всі

оператори, що розташовані перед оператором wait, після чого процес буде припинений.

Лістинг базового VHDL-коду:

```
library IEEE;
use IEEE.std_logic_1164.all;
entity element is
    port(
        CLK : in STD_LOGIC;
        Ainout : inout STD_LOGIC;
        Bout : out STD_LOGIC
    );
end element;

architecture proc of element is

begin
    Pr_CLK: process(CLK)
    begin
        end process Pr_CLK;
    Pr_A: process(CLK)
    begin
        if clk'event and CLK ='1' then Ainout<='1' after 5 ns;
            elsif clk'event and CLK='0' then Ainout<='0' after 5 ns;
        end if;
    end process Pr_A;
    Pr_B: process (Ainout)
    begin
        if Ainout'event then Bout<= not Ainout;
        end if;
    end process Pr_B;
end proc;
```

Виконуємо компілювання створеного об'єкту та запускаємо процес симуляції декодера (див. рис. 2).

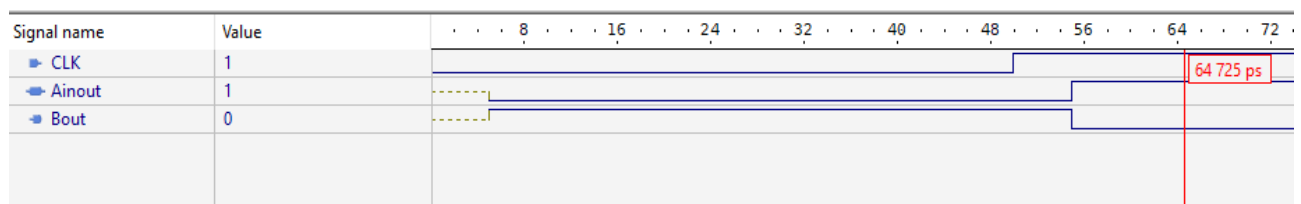


Рисунок 1 – Часова діаграма роботи базового коду

Модифікація А базового коду – за допомогою оператора wait for формуємо синхронізуючий сигнал CLK в тілі процесу Pr_CLK. Рівень цього сигналу повинен змінюватись кожні 10 ns. Процес Pr_A переписуємо з використанням оператора wait on, а процес Pr_B – з використанням оператора wait until.

Лістинг модифікованого VHDL-коду v.A:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity elementA is
    port(
        Ainout : inout std_logic;
        Bout : out std_logic
    );
end elementA;

architecture elementA of elementA is
    signal CLK : std_logic := '0';
begin
    Pr_CLK: process
    begin
        while true loop
            CLK <= not CLK;
            wait for 10 ns;
        end loop;
    end process Pr_CLK;
    Pr_A: process
    begin
        wait on CLK;
        if CLK = '1' then
            Ainout <= '1' after 5 ns;
        elsif CLK = '0' then
            Ainout <= '0' after 5 ns;
        end if;
    end process Pr_A;
    Pr_B: process
    begin
        wait until Ainout'event;
        Bout <= not Ainout;
    end process Pr_B;
end elementA;
```

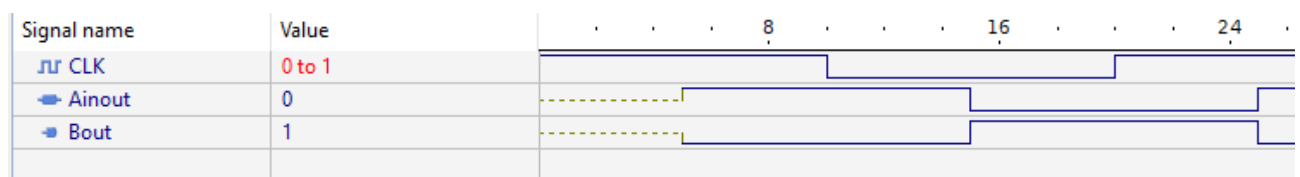


Рисунок 2 – Часова діаграма роботи модифікованого коду v.A

Модифікація В – додаємо до послідовності операцій процесу Pr_A вираз wait for 30 ns.

Лістинг модифікованого VHDL-коду v.B:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity elementB is
    port(
        Ainout : inout std_logic;
        Bout : out std_logic
    );
end elementB;

architecture lab4b of lab4b is
    signal CLK : std_logic := '0';
begin
    Pr_CLK: process
    begin
        while true loop
            CLK <= not CLK;
            wait for 10 ns;
        end loop;
    end process Pr_CLK;
    Pr_A: process
    begin
        wait on CLK;
        if CLK = '1' then
            Ainout <= '1' after 5 ns;
            wait for 30 ns;
        elsif CLK = '0' then
            Ainout <= '0' after 5 ns;
            wait for 30 ns;
        end if;
    end process Pr_A;
    Pr_B: process
    begin
        wait until Ainout'event;
        Bout <= not Ainout;
    end process Pr_B;
end elementB;
```

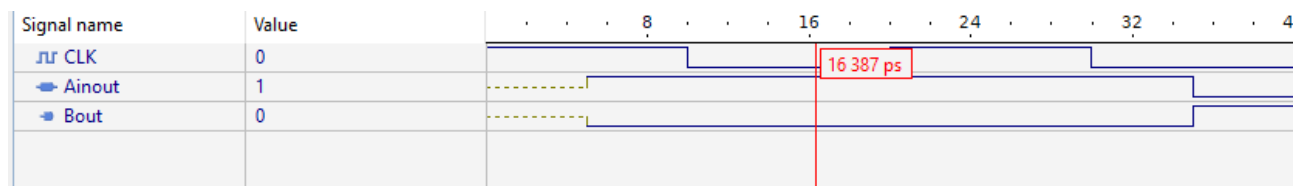


Рисунок 3 – Часова діаграма роботи модифікованого коду v.B

Висновки: в процесі виконання лабораторної роботи було отримано практичні навички застосування оператора wait при моделюванні різних режимів очікування у Active-HDL та виконано порівняння часових діаграм.